

Appunti Post pausa. Ore 14.00 Bella copia

Ecco gli appunti riscritti in forma discorsiva e professionale.

Il pattern Interceptor-Filter per l'autenticazione JWT

Il diagramma descrive il flusso di comunicazione tra un client e un server, con particolare attenzione alla gestione della sicurezza tramite JWT. In termini tecnici, questo schema rappresenta il pattern Interceptor-Filter per l'autenticazione in un'architettura REST.

Il processo inizia quando l'applicazione deve inviare una richiesta al server. Il client genera la richiesta HTTP, ma prima che essa lasci il dispositivo, passa attraverso un interceptor, il cui compito è iniettare automaticamente il JWT nell'header della richiesta sotto la chiave `Authorization: Bearer <token>`. La richiesta esce quindi dal client arricchita con il token di sicurezza.

Lato server: la catena di filtri

Una volta che la richiesta arriva al backend, non raggiunge immediatamente i dati, ma deve superare una serie di controlli. Il filtro è il primo punto di ingresso: legge l'header per verificare se il JWT è presente e strutturalmente valido. Successivamente avviene la fase di autenticazione, in cui il server convalida la firma del token; se il token è valido e non è scaduto, l'utente viene considerato autenticato. Si passa poi alla fase di autorizzazione, in cui il sistema verifica se l'utente possiede i permessi, ovvero i ruoli, necessari per accedere alla risorsa richiesta. Solo se tutti i passaggi hanno esito positivo la richiesta raggiunge il controller, che elabora i dati e restituisce la risposta al client. Se il JWT fosse assente o non valido, il filtro bloccherebbe la richiesta immediatamente con un errore `401 Unauthorized`, senza coinvolgere in alcun modo l'API.

Perché questo modello è rilevante

Il server non ha bisogno di sessioni né di salvare i dati dell'utente in memoria, poiché tutte le informazioni necessarie sono contenute nel JWT: il sistema è stateless per definizione. Questo garantisce una netta separazione delle responsabilità, poiché l'API non si occupa di verificare l'identità del chiamante, lavoro svolto interamente a monte dai filtri. Grazie all'interceptor, lo sviluppatore non deve aggiungere manualmente il token ad ogni chiamata, riducendo il rischio di errori.

La Guard lato client

In Angular è possibile indicare a una pagina di non poter essere caricata se l'utente non è loggato. Questo controllo non serve a garantire la sicurezza dei dati, ma a migliorare l'esperienza dell'utente ed è aggirabile. Quando l'utente tenta di navigare verso una rotta protetta, Angular non carica immediatamente il componente, ma interroga la Guard: se l'utente non è loggato, la navigazione viene annullata e l'utente viene reindirizzato forzatamente alla pagina di login, senza che il browser si ricarichi mai completamente. La rotta `/login` deve essere necessariamente libera, altrimenti nessun utente potrebbe mai raggiungere il form di autenticazione.

La doppia barriera di difesa

Il sistema implementa una strategia di difesa a due livelli distinti. Lato server, le risorse sono classificate in base alla loro visibilità: `/login` è pubblico e accessibile a tutti, mentre risorse come `/guests` sono private e richiedono un JWT valido; l'obiettivo è garantire l'integrità dei dati. Lato client, la Guard gestisce l'esperienza di navigazione, impedendo all'utente di vedere pagine vuote o non autorizzate e reindirizzandolo correttamente; l'obiettivo è la qualità dell'interfaccia utente.

Nel progetto SBB il lato server è già protetto: anche lasciando un componente Angular libero, il server rifiuterebbe comunque la richiesta, ma non sarebbe una buona esperienza utente. Non si può mai fare affidamento esclusivamente sulle Guard del client, poiché un utente esperto potrebbe disattivarle tramite la console del browser. La vera sicurezza risiede sempre nella configurazione del server.

Autorizzazione basata sui ruoli

Il controllo degli accessi non si limita a verificare se un utente è loggato, ma deve anche stabilire cosa può fare, ovvero l'autorizzazione basata sui ruoli. Una Guard semplice verifica solo la presenza di un JWT valido, mentre una Guard evoluta estrae il ruolo dal payload del token — ad esempio `role: 'RECEPTIONIST'` — e lo confronta con quello richiesto dalla rotta. In Angular è possibile passare informazioni aggiuntive alla rotta tramite il campo `data`, specificando quale ruolo è necessario per accedere a quel percorso.

La catena di sicurezza completa funziona su due livelli concorrenti. Lato client, la Guard legge il ruolo dal token e, se insufficiente, blocca la navigazione reindirizzando l'utente. Lato server, se l'utente forzasse l'URL o bypassasse il codice JavaScript, Spring Security verificherebbe comunque il JWT allegato alla richiesta e, riscontrando un ruolo insufficiente, risponderebbe con un errore `403 Forbidden`. La sicurezza lato client serve esclusivamente per l'esperienza utente, evitando di mostrare pagine o pulsanti non pertinenti; la protezione reale dell'integrità dei dati avviene sempre e soltanto sul server, che non si fida mai ciecamente del client.